IEICE TRANS. INF. & SYST., VOL.E108–D, NO.4 APRIL 2025

PAPER

Multi-Grained Guaranteeable Requirement Analysis for Iterative Adaptation

Jialong LI[†a)], **Takuto YAMAUCHI**[†b)], **Takanori HIRANO**[†], **Jinyu CAI**[†], ***Nonmembers*, *and*** **Kenji TEI**[††c)], ***Member***

SUMMARY In the studies of self-adaptive systems (SAS), requirement relaxation is a well-studied approach to adjust or disable certain requirements in response to requirement unsatisfaction or requirement conflicts, allowing the system to maintain core functionalities while temporarily reducing service quality. The recent integration of Guaranteeable Requirement Analysis (GRA) with Discrete Controller Synthesis (DCS) allows for coordinated self-adaptation by identifying relaxable requirements and then synthesizing new specifications to fulfill remaining requirements. However, the scalability of GRA poses challenges, particularly due to state explosion and combination explosion, making it difficult to apply to runtime self-adaptation due to timeliness reasons. To address this, this paper introduces the Multi-grained Guaranteeable Requirement Analysis (MGRA) approach, which (i) employs a multi-round adaptation process to deal with environmental changes and (ii) controls the trade-off between computation time and adaptation quality by adjusting the granularity of analysis. More specifically, the adaptation starts with a quick, coarser GRA for an initial adaptation to meet timeliness, followed by iterative refinements for finer GRA with higher-quality adaptations to meet more requirements gradually. The applicability and effectiveness have been assessed through two case studies.

key words: *requirement satisfiability, RE@runtime, self-adaptive systems, controller synthesis*

1. Introduction

Software is increasingly becoming integral to our society. Along with this integration, software faces escalating uncertainties due to the complex and open nature of the real-world operational environment. In response to these uncertainties, there is a significant and growing interest in self-adaptive systems (SAS) that are capable of autonomously adapting to changes in their environment.

In SASs, adapting system configuration and architecture stand as the predominant approaches to maintaining safety and enhancing performance amidst uncertainties. However, there is also a growing advocacy within the self-adaptive systems and requirements engineering communities to consider system requirements, i.e., the objectives of the system, as targets for adaptation [1]. With the incorporation of the requirement adaptation, SASs can achieve a higher degree of flexibility and resilience, thereby enhancing its ability to navigate complexity and uncertainty [2].

In studies of requirement adaptation, the most commonly explored approach is requirement relaxation, also known as requirement degradation. This involves relaxing or disabling certain requirements to facilitate a controlled reduction in service quality. Generally, a requirement is relaxed under two types of situations: (i) When a requirement becomes unsatisfiable due to environmental changes. For instance, if a third-party service becomes unavailable, services dependent on it cannot be fulfilled. In such cases, the related service might be temporarily suspended (i.e., relaxing the related requirement), allowing the system to continue providing other services (i.e., fulfilling the remaining requirements). (ii) In cases of requirement conflict caused by environmental changes. An example is the scenario with a platform like X (Twitter), where, during a surge in user traffic leading to server resource shortages, certain functions like search might be temporarily disabled to ensure the availability of core features, such as tweeting [3].

The integration of Guaranteeable Requirement Analysis (GRA) with Discrete Controller Synthesis (DCS) has been explored recently to facilitate self-adaptation through requirement relaxation [4]. This combination enables SAS to achieve coordinated adaptation at both the requirement and behavioral levels. When unforeseen environmental changes occur, GRA is first applied to determine which requirements—either those that are unsatisfiable or those of lower priority in conflict situations—can be relaxed. This determination is made by evaluating the satisfiability of all possible requirement combinations [5], [6]. Subsequently, DCS is utilized to synthesize a new behavioral-level specification, ensuring that the system can fulfill the requirements that remain post-relaxation [7], [8].

However, a fundamental challenge in applying GRA is its scalability for runtime use. Specifically, the analysis time required often increases exponentially with the number of requirements, due to two types of explosions as outlined below: (i) Combination explosion: For N requirements, there are 2^N possible combinations for relaxation. (ii) State explosion: The GRA analysis space should include the state-action space after requirement relaxation to enable further analysis of the remaining requirements. As the requirement combination explodes, the analysis space, that corresponds to different subsets of remaining requirements, explodes accordingly. Therefore, this makes it challenging to employ GRA, especially for use in runtime adaptation, since it often demands timeliness to deal with environmental changes.

To address these challenges, this paper introduces an approach, namely Multi-grained Guaranteeable Requirement

Manuscript received August 2, 2024.
Manuscript publicized October 31, 2024.
[†]Waseda University, Tokyo, 169–8050 Japan.
[††]Tokyo Institute of Technology, Tokyo, 152–8550 Japan.
a) E-mail: lijialong@fuji.waseda.jp
b) E-mail: itaku3@akane.waseda.jp (Corresponding Author)
c) E-mail: tei@c.titech.ac.jp
DOI: 10.1587/transinf.2024EDP7200

Copyright © 2025 The Institute of Electronics, Information and Communication Engineers

Analysis (MGRA). MGRA is built upon two key concepts: (i) Within the context of GRA, we state that it is possible to control the trade-off between computation time and adaptation quality by adjusting the granularity of analysis. In other words, the above two explosions can be effectively mitigated by increasing the granularity of analysis, while the cost is a reduction in adaptation quality; (ii) Utilizing this property, we further propose MGRA. Unlike previous approaches that adapt the system only once per environmental change, our MGRA allows the system to adapt multiple times for each environmental change, thereby gradually improving adaptation quality and fulfilling a greater number of requirements. Specifically, MGRA employs a multi-round approach to GRA, beginning with a rapid but coarser GRA for initial adaptation to mitigate the scalability problem and meet adaptation timeliness. Subsequent iterations of GRA become increasingly refined, and gradually enhance the adaptation quality to meet more requirements (i.e., relax fewer requirements) as the granularity of the analysis is reduced. This iterative process allows for more precise analysis and higher-quality adaptation over time.

The initial concept of this paper was introduced in prior literature [6]. This paper expands upon that study in the following ways:

- We declare the specific terms of analysis granularity, and adaptation quality in the context of GRA, and state the premise of MGRA, i.e., with the increase in analysis granularity level, computation time decreases (i.e., the explosions are mitigated), and adaptation quality also decreases (Sect. 3). Specifically, while the concepts of the terms proposed in this paper are similar to those in [6], we have refined these terms to enhance both accuracy and understandability, and have provided a more detailed explanation.
- We detail the technical aspects of MGRA, including the design methodology of multi-grained analysis at design time and the process of iterative adaptation at runtime (Sect. 4). Specifically, we further elaborate on the methodology of multi-grained design, such as the constraints that need to be considered.
- We conduct an evaluation using two applications and engage in further discussion (Sect. 5). Specifically, we carry out an additional experiment on a new application and offer a more detailed discussion concerning the limitations, factors influencing computation time, and potential threats to validity.

The structure of this paper is as follows: Sect. 2 provides the necessary background. Section 3 offers a statement of the relations among analysis granularity, analysis time, and adaptation quality. Building on this, Sect. 4 delves into the detailed explanation of MGRA. Section 5 evaluates and discusses MGRA through two case studies. Section 6 reviews related work in the field. Finally, Sect. 7 concludes the paper with a brief summary and outlines future work.

2. Background

This section offers the necessary background information related to GRA and DCS. Figure 1 depicts the relationships between each of the models, and Fig. 2 provides an illustrative example to support the ensuing explanation. The subsequent parts of this section will introduce each element of Fig. 1 individually.

2.1 Labelled Transition Systems

Definition 1. *(Labelled Transition Systems (LTS)) An LTS is $E = (S, L, \Delta, s_0)$ where S is a finite set of states, $L = L_C \cup L_M$ is a finite set of labels, L_C is a set of controllable labels, L_M is a set of monitorable labels, and $L_C \cap L_M = \emptyset$. $\Delta = S \times L \times S$ is a set of transition relations, and $s_0 \in S$ is the initial state. An execution trace of E is $t = s_0, l_0, s_1, l_1, \ldots$, which follows Δ.*

The Environment Model (E) presents assumptions about interactions between the system and its operational environment [9]. For example, if a robot picks up products and then moves, the products' location changes [10]. The illustration in Fig. 2-(a) shows that the system can execute actions A or B, returning to the initial state after each action.

The Tester Model (T), derived from the requirements (R) that is formalized as linear temporal logic (LTL) [11], tracks requirement compliance. This special LTS transitions to a state of -1 upon requirement violation. Our focus is on safety properties, ensuring that 'bad things never happen' [12]. The examples illustrated in Fig. 2-(b) and (c) represent the following requirements: R1: Action A should

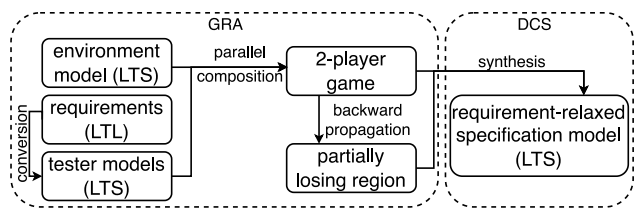


Fig. 1 Overview of Background

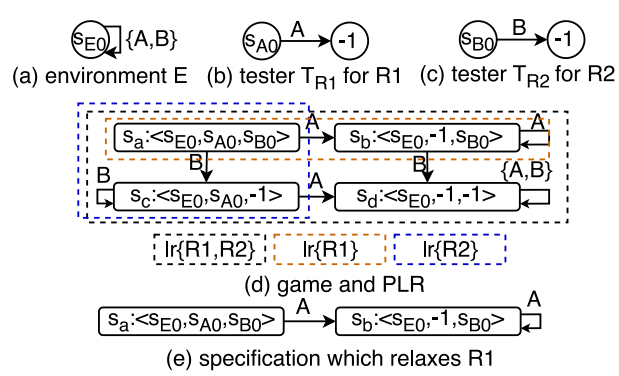


Fig. 2 Illustrative example for introducing background. For simplicity, all transitions are controllable.

never be executed; R2: Action B should never be executed.

The Specification Model ($Spec$), synthesized from the environment and tester models via DCS, describes the software system's behavior. Adhering to the world-machine model proposed by Michael Jackson in requirements engineering [9], the condition $E \parallel Spec \models R$ holds true, meaning the specification $Spec$, within the operational environment E, satisfies the requirements R.

Regarding the system operation, it uses an LTS-based specification model to track the current state [13]. When the system is in a controllable state, it selects and executes a controllable action. In contrast, when in a monitorable state, the system waits to receive events that are either monitored or returned. As either an action is executed or an event is received, the system's current state transitions accordingly. Interested readers can refer to [13] for more detailed information.

2.2 Two-Player Game

Definition 2. *(Parallel Composition) Given an environment model $E = (S_E, L_E, \Delta_E, s_{E0})$ and tester model $T = (S_T, L_T, \Delta_T, s_{T0})$ such that $L_T \subseteq L_E$. A parallel composition $E \parallel T$ is the LTS $E \parallel T = (S, L, \Delta, s_0)$, where $S \subseteq S_E \times S_T, L = L_E, s_0 = (s_{E0}, s_{T0})$, and Δ is the smallest relation that satisfies the rules below.*

$$\frac{s_E \xrightarrow{\ell} s'_E,\ s_T \xrightarrow{\ell} s'_T}{(s_E, s_T) \xrightarrow{\ell} (s'_E, s'_T)}\ell \in L_E, \quad \frac{s_E \xrightarrow{\ell} s'_E,\ s_T \xrightarrow{*} s_T}{(s_E, s_T) \xrightarrow{\ell} (s'_E, s_T)}\ell \in L_E \backslash L_T$$

Parallel composition is a technique used to integrate separate LTS models into a unified LTS model [14]. In the context of GRA and DCS, parallel composition is utilized to create a two-player game, which will be elaborated upon in a later section. When dealing with multiple models, the process of parallel composition is defined inductively; for instance, $E \parallel T_1 \parallel T_2 = ((E \parallel T_1) \parallel T_2)$.

Definition 3. *(Two-player Game) A two-player game (hereafter game) is $G = (S_g, L, \Gamma, s_{g_0}, X)$, where S_g is a finite set of states, $L = L_C \cup L_M$ is a finite set of labels, $\Gamma^{-/+} \subseteq S_g \times L_M/L_C \times S_g$ is a finite set of monitorable/controllable transition relations, $s_{g_0} \in S_g$ is the initial state, and $X = \{x_{\phi_i}\}$ is a family of sets over S_g that stores the losing conditions and be indexed by a requirement set ϕ_i. A state s_g is a monitorable state if $\Gamma^-(s_g) \neq \emptyset$, and controllable otherwise. A play on G is a sequence $p = s_{g_0}, s_{g_1}, \ldots$ that follows $\Gamma^- \cup \Gamma^+$. If plays contain a state $s_x \in x_{\phi_i}, x_{\phi_i} \in X$, they are losing for the controller in G.*

A two-player game (hereafter game) serves as the analysis space for both DCS and GRA. The game involves two players: (i) the system, which selects controllable transitions to execute in controllable states that only have controllable transitions, and (ii) the operational environment, which decides which monitorable transitions occur in monitorable states, where at least one monitorable transition exists. Figure 2-(d) illustrates an example that shows a game composed of E, $T_{\{R1\}}$, and $T_{\{R2\}}$.

2.3 Partially Losing Region

Definition 4. *(Partially Losing Region (PLR)) Given game $G = (S_g, L, \Gamma, s_{g_0}, X)$, the $PLR = \{lr_{\phi_i}\}$ is a family of sets over S_g and be indexed by a requirement set ϕ_i. From a state $s \in lr_{\phi_i}, lr_{\phi_i} \in PLR$, the system can always reach a state $s_x \in x_{\phi_{i_{sub}}}, x_{\phi_{i_{sub}}} \in x_{\phi_i}, x_{\phi_i} \in X$ for game $G' = (S_g, L, \Gamma, s_{g_0}, x_{\phi_{i_{sub}}})$.*

In this paper, the term GRA is used to describe the process of accurately identifying PLR. PLR is used to identify potential violations across all combinations of requirements. Each losing region, denoted as $lr_{\phi_i} \in PLR$, represents a region where a specific subset of requirements cannot be guaranteed. For a state that falls into multiple regions, this indicates that the system has several options for deciding which set of requirements to relax.

Consider, for example, two requirements R1 and R2. There are three types of losing regions: lr_{R1}, lr_{R2}, and $lr_{R1,R2}$. In the example shown in Fig. 2-(d), state s_a, which belongs to lr_{R1}, can guarantee all other requirements (i.e., R2) if it relaxes R1. A state belonging to different regions presents multiple relaxation options, as illustrated in Fig. 2-(d), where the system has three relaxation choices: R1, R2, and R1, R2.

PLR is derived through backward propagation, which involves extending regions from losing conditions (the states that directly violate the requirement) in the direction opposite to transitions [15]. The propagation rule varies between controllable and monitorable states. In a controllable state, the controller can choose which transition to execute, implying the system has options regarding which set of requirements to violate. Hence, the state is defined as belonging to all losing regions of its subsequent states (e.g., s_a belongs to three regions). In a monitorable state, where the controller cannot select the occurring transition, the losing region of the target state is defined as the union of its subsequent states' losing regions. For instance, if actions A and B are monitorable, all states in Fig. 2-(d) would belong exclusively to $lr_{R1,R2}$.

With the game setup and PLR identified, requirement relaxation decisions are made by comparing the priorities of the conflicting requirements. For instance, if the system is at state s_a and R2 is of higher priority than R1, the system opts to relax R1 while ensuring R2 is satisfied. Consequently, the synthesis of a specification model can be realized by extracting the state-action spaces from the game that comply with the remaining, unrelaxed requirements, as exemplified in Fig. 2-(e). We omit the technical details of relaxation and synthesis, as they are not our primary focus. Interested readers seeking more information can refer to [8], [16].

3. Premise of MGRA

In this section, we dive into the definition of specific terms in the context of GRA (i.e., analysis granularity, analysis time, adaptation quality), followed by an explanation of their

Table 1 The impact of analysis granularity on analysis time, and adaptation quality

Analysis Granularity	Game Space	PLR Combinations	Analysis Time	Adaptation Quality
High	Small	Little	Short	Low
Low	Large	Much	Long	High

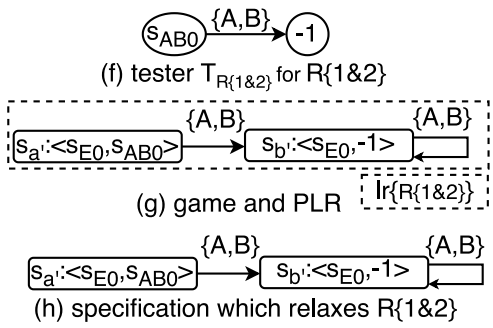


Fig. 3 Illustrative example of a higher-grained GRA

relationship. A brief summary of the discussion in this section is illustrated in Table 1.

3.1 Analysis Granularity

In the context of this paper, analysis granularity refers to the degree of detail applied during the analysis and relaxation of requirements within GRA. We especially note that the term "granularity" in our paper is distinct from the granularity of modeling in states or actions that are commonly discussed in existing studies on model checking and runtime verification. Specifically, the requirements used in a higher granularity-level analysis should be the AND conjunction of the requirements in a lower granularity-level analysis. For instance, considering requirements R1 and R2, Fig. 2 approaches them as distinct entities, evaluating their satisfiability either individually or collectively. This condition is considered an analysis of low granularity. Conversely, Fig. 3 aggregates R1 and R2 into a consolidated unit, R1&2, for the purposes of analysis and relaxation. Here, given R1 ($\Box\neg A$) and R2 ($\Box\neg B$) represented in LTL, R1&2 formulated as '$\Box(\neg A \wedge \neg B)$' represents the logical AND-conjunction of R1 and R2, implying that 'Neither action A nor B should be executed' (Fig. 3-(f)). When R1&2 is relaxed, it implies a simultaneous relaxation of both R1 and R2 (i.e., the specification depicted in Fig. 3-(h) fails to satisfy neither R1 nor R2). This scenario is considered to have high analysis granularity.

3.2 Analysis Time

In this paper, analysis time specifically refers to the time required to identify PLR. As analysis granularity increases, the reduction in analysis time is primarily due to the mitigation of state explosions and the explosion of requirement combinations.

State explosion refers to the increase in the number of states within the game, an intrinsic nature of GRA. For example, the comparison between Fig. 2-(d) and Fig. 3-(g) illustrates this concept. The necessity to analyze the state space even after some requirements are violated arises for two reasons: (i) In GRA, analyzing these states is crucial to determine possible relaxations, such as confirming whether 'R2 can be guaranteed if R1 is relaxed' or 'R2 will also be violated even if R1 is relaxed' (Fig. 2-(d)); (ii) In DCS, these states may contribute to synthesizing a requirement-relaxed controller, as they might be part of the controller that satisfies the remaining requirements. For example, s_b is part of the R1-relaxed controller because such a state can guarantee the remaining requirement set {R2}. With the increase in analysis granularity, state explosion will be suppressed because the AND-conjunction of requirements inherently abstracts the states after requirement violation [17]. Given two requirement tester models T_{R1} and T_{R2}, their parallel composition with the environment model results in the game shown in Fig. 2-(d). For this game, by abstracting all states that violate either R1 or R2 (s_b, s_c, s_d) into a single state, it results in the game shown in Fig. 3-(g).

The requirement combination refers to the partially losing regions of different requirement combinations in PLR. For example, in Fig. 2-(d), there are three different losing regions [17]. Given N requirements and an additional requirement that the system should not reach deadlock, a state may belong to up to $2^{(N+1)} - 1$ different losing regions. With the increase in analysis granularity, the requirement combination will naturally be mitigated, as the number of requirements for analysis will decrease due to the AND conjunction.

3.3 Adaptation Quality

Aligning with many previous studies, adaptation quality is quantified using a utility function that calculates the weighted sum of the satisfaction level for each requirement [18]. In the context of GRA, the satisfaction level of requirements is assessed as binary (either satisfied or not), rather than quantitative or probabilistic in existing studies. Thus, when a requirement is satisfied post-relaxation, its satisfaction level is considered to be 1; otherwise, it is 0. For simplicity, this paper assumes that each requirement in the lowest granularity analysis has a weight of 1—for example, the weights of R1 and R2 are each 1. At higher granularity levels, the weight of a combined requirement equals the sum of the weights of its constituent requirements. Consequently, the combined requirement R1&2 has a weight of 2, which is the sum of the weights for R1 and R2.

As the granularity level becomes lower, the adaptation quality may improve, depending on the current state of the system. Consider Fig. 3-(g), where the adaptation quality at state s'_b is 0. If the current state of the system corresponds to s_b in Fig. 2-(d), then the adaptation quality could increase to 1, reflecting the guaranteed satisfaction of R1. However, if the current state is s_d, the adaptation quality would remain at 0. It should be noted that while finer granularity in analysis doesn't always enhance adaptation quality, it will not reduce

it either, which is attributed to the game-space abstraction that results from the AND-conjunction of requirements.

4. Multi-Grained Guaranteeable Requirement Analysis

Given the premise discussed in the last section, this section begins by providing an overview of our proposal. Following this, we delve into the technical details of MGRA, covering both the design-time phase and the runtime phase.

4.1 Overview

Figure 4 presents an overview of the differences between the existing baseline approach and our MGRA approach. Both approaches encompass two phases: (i) a design phase where system designers are required to delineate requirements, setting the foundation for the runtime phase; and (ii) a runtime phase that implements the GRA and generates a requirement-relaxed specification as an adaptation solution. However, as illustrated in Fig. 4 - (b), our MGRA approach incorporates an iterative adaptation process with varying levels of analysis granularity. This process initiates with the GRA and the subsequent DCS at the highest granularity level (steps 1 and 2), proceeds through intermediate iterations that gradually transition to finer granularity (steps 3-5), and concludes with the iteration at the lowest granularity level (steps 6-8).

4.2 Design-Time Phase: Multi-Grained Design

During the design-time phase, designers or engineers are tasked with structuring requirements across multiple levels of granularity. It is important to clarify that the objective of this phase is not to elicit or elaborate unknown requirements, such as through goal model construction for requirement elicitation. Rather, this phase focuses on designing the unit for analysis and relaxation at different levels of granularity, based on the assumption that both the environment model and the system's requirements are already known.

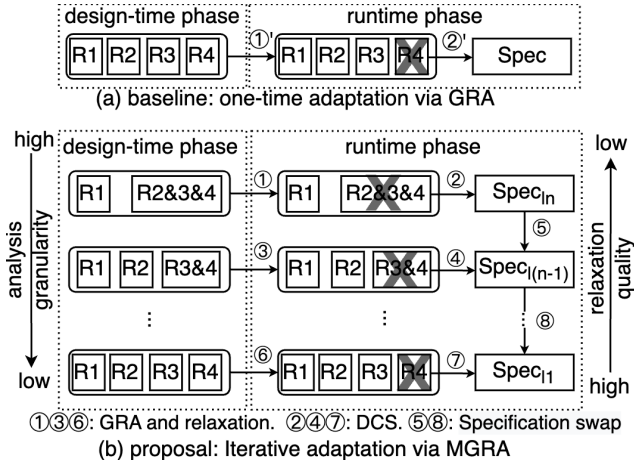


Fig. 4 Comparison of existing baseline approach and our MGRA approach. Fork indicates the requirement that is relaxed.

We outline considerations for designing multi-grained analysis. Key points include: (i) Relationship of requirements at different levels: We advocate that higher-level requirements should be structured as AND-conjunctions of lower-level requirements or as the lower-level requirements themselves. Here, this constraint is to meet the premise mentioned in Sect. 3, thereby ensuring the relationship between analysis time and adaptation quality always holds at different levels. However, MGRA theoretically allows flexibility in how requirements are structured as long as the analysis time at upper layers is faster than at lower layers, for instance, the top-level requirement could also be R{1&2} and R{3&4} in Fig. 4; (ii) Modeling order: While MGRA does not mandate a specific order for requirement modeling, starting from the lowest level to higher levels is often practical. This approach facilitates combining requirements using AND-conjunctions, which is typically simpler than decomposing complex requirements into finer details; (iii) Design of the highest level: The highest level of design should take into account the application's tolerance for adaptation or response time. The granularity at this level directly influences the initial adaptation time. Applications that can accommodate longer response times may allow for more detailed granularity at the highest level. Moreover, this level could consider combining all requirements into a single comprehensive requirement, thereby assessing the combined satisfiability of all requirements. This design strategy is particularly useful in scenarios where, despite environmental changes, all requirements remain fulfillable, and thus, relaxation is unwarranted. In such a scenario, it could expedite the GRA analysis, allowing for an immediate swap to a new specification model that satisfies all requirements; (iv) Design of the lowest level: The design at the lowest level, representing the finest granularity, must account for the frequency of environmental changes, as it is necessary to re-analyze (from the highest level) once the new environment change occurs. For example, if the environment changes each hour but the analysis at the lowest level takes 10 hours, then the analysis at the lowest level is meaningless; (v) Middle-level requirements and level count: The design of middle-level requirements and the determination of the number of granularity layers should account for the cost of specification switching in the application context [19]. Executing numerous iterations in a brief timeframe may be impractical if each adaptation process incurs substantial switching overhead.

4.3 Runtime Phase: Iterative Adaptation

Figure 5 illustrates a brief overview of an SAS based on the MAPE (Monitor-Analyze-Plan-Execute) conceptual architecture [20]. The SAS is divided into two principal components: the managing system, which is responsible for performing MGRA and re-synthesizing the specifications for the managed system, and the managed system, which is controlled to interact with the environment in accordance with the given specifications.

Algorithm 1 delineates the detailed procedure within the managing system, embodying the MAPE-loop illustrated in Fig. 5. The inputs of the algorithm are: E, denoting the environment model; E_{df}, representing changes (i.e., difference) in the environment model; $\{G_{lv_i}\}_{lv_i \in L}$, a set of games G indexed by different granularity levels lv_i; $\{PLRlv_i\}_{lv_i \in L}$, a set of PLRs indexed by different granularity levels lv_i. Initially, the algorithm sets the granularity level, denoted as lv_{cnt}, to the highest level, lv_n(line 2). Upon detecting an environment change, that indicates a discrepancy between the existing environment model and the real world (e.g., an unforeseen event), the following process will be performed at each granularity level: Firstly, the monitor updates the environment model to reflect the observed discrepancy (e.g., by adding a new monitorable transition for the event) [21]. Subsequently, the game model is also updated to correspond to the change in the environment model (line 4). Secondly, the analyzer is responsible for performing GRA using game models, with each GRA at different levels conducted independently (line 5, corresponds to steps 1, 3, and 6 in Fig. 4-(b)). Thirdly, the planner determines the requirement relaxation based on GRA outcomes (line 8), and then generates a relaxed specification model $Spec_{lv_i}$ (line 9, corresponds to steps 2, 4, and 7 in Fig. 4-(b)). Here, the conditional judgment is to avoid iterations from lower to higher granularity levels (line 6). Finally, the executor is responsible for swapping to the new specifications of the managed system (line 10, corresponds to steps 5 and 8 in Fig. 4-(b)). In this study, we assume that the specification swap occurs when the managed system reverts to its initial state (e.g., a robot completes a transportation task and returns to the charging station). That is also the reason why we utilize the initial state s_{E0} at lines 8 and 9.

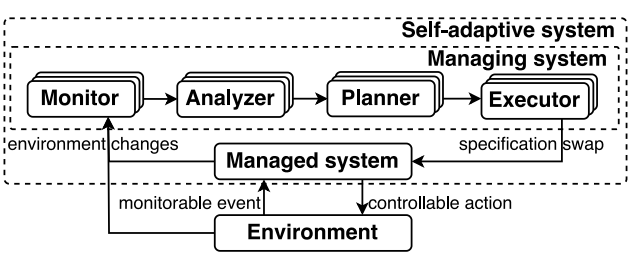


Fig. 5 Overview of the SAS. The multiple instances of MAPE refer to ones at different granularity levels.

Algorithm 1 Procedure of iterative adaptation with MGRA

1: **Input:** $E, E_{df}, \{G_{lv_i}\}_{lv_i \in L}, \{PLR_{lv_i}\}_{lv_i \in L}$
2: $l_{cnt} = l_n$
3: **for** $\{lv_i = \{n, n-1, \ldots, 2, 1\}\}$ **do**
4: $\quad E, G_{lv_i} \leftarrow \text{Update}(E, G_{lv_i}, E_{df})$ ▹ M
5: $\quad PLR_{lv_i} \leftarrow \text{GRA}(G_{lv_i})$ ▹ A
6: $\quad$ **if** $lv_i \leq l_{cnt}$ **then**
7: $\quad\quad l_{cnt} = lv_i$
8: $\quad\quad R_{lv_i} \leftarrow \text{Relaxation}(PLR_{lv_i}, s_{E_0})$ ▹ P
9: $\quad\quad Spec_{lv_i} = \text{DCS}(G_{lv_i}, PLR_{lv_i}, R_{lv_i}, s_{E_0})$ ▹ P
10: $\quad\quad \text{Swap}(Spec_{lv_i})$ ▹ E
11: $\quad$ **end if**
12: **end for**

For the above algorithm, two points warrant attention: (i) In line 3, the analysis across all granularity levels can proceed sequentially (i.e., from high to low granularity) or in parallel, contingent upon the computation resources available. Even with parallel processing, the iterative nature of our method is preserved, as computation at higher granularity levels typically concludes more swiftly; (ii) If a change in the environment emerges during the algorithm's execution, the ongoing computation (specifically, the analysis at a finer granularity prior to the environment change) will cease, and a new MGRA will commence for the new environment conditions, starting at the highest level of granularity.

5. Evaluation

To evaluate MGRA, we set the following two research questions.

- RQ1 (Applicability): Can MGRA be applied to different domain applications? Specifically, RQ1 seeks to validate whether the design of MGRA instricts the applicability range of GRA, prior to examining scalability issues.
- RQ2 (Effectiveness): How do the changes in granularity level influence the trade-off between computation time and adaptation quality? Specifically, what extent of reduction in the scale of the state space and analysis time is achieved at each granularity level (i.e., the effectiveness degree of mitigating the scalability issue)? Furthermore, how many requirements can be guaranteed at each respective level of granularity, and how many additional computation costs are required for MGRA (i.e., the cost of mitigating the scalability issue)?

5.1 Applicability – RQ1

Regarding RQ1, we apply MGRA to two applications–a warehouse robot system and a production cell system, as shown in Fig. 6 – to demonstrate the applicability of MGRA. Due to space constraints, we only provide a brief description of these two applications in the rest of this section; interested readers can check the detailed modeling of environments and requirements through the link: https://github.

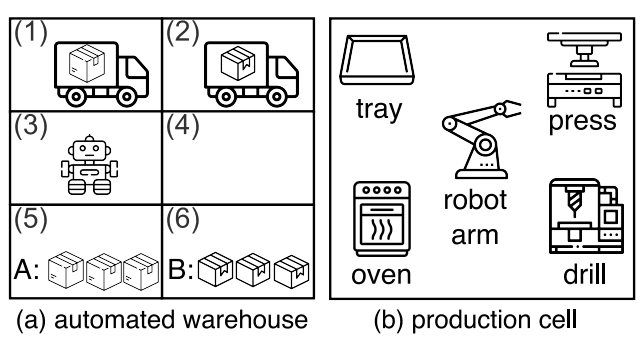


Fig. 6 Two applications for evaluation.

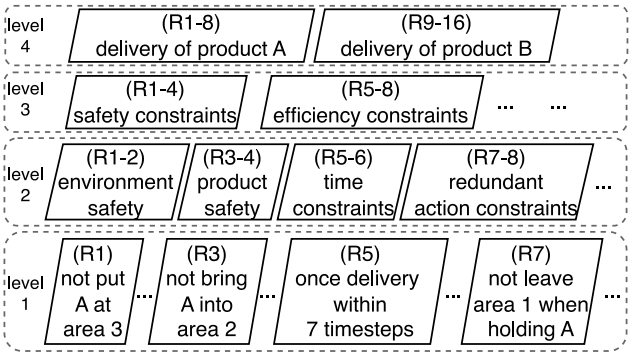


Fig. 7 Multi-grained requirement design for warehouse

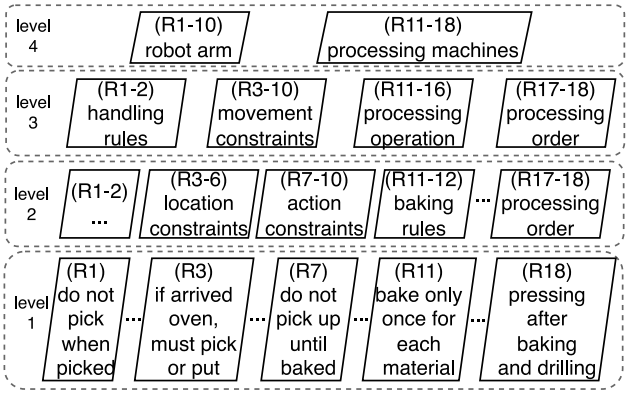


Fig. 8 Multi-grained requirement design for production cell

com/545659928/MGRA.

5.1.1 Warehouse Robot System

The first application involves a warehouse robot system responsible for transporting products within a warehouse, as illustrated in Fig. 6-(a). The system includes a robot and various designated areas such as shipping areas, passageway areas, and storage areas. Storage areas 5 and 6 store different products, A and B, respectively, which the robot must deliver to their corresponding shipping areas 1 and 2. The environment model of this application captures the potential interactions among the robot, the products, and the warehouse. For example, when the robot executes controllable actions such as `move_up` or `pick_up`, this results in observable monitorable actions like `arrived[1]` or `pick_success`.

For the system requirement, Fig. 7 shows the multi-grained requirements we designed, where each requirement level corresponds to different analysis granularity levels. The requirement numbering illustrates how each requirement is composited; for example, R1-2 is a combination of R1 and R2. To conserve space, ellipses are used to omit requirements that are similar. For instance, (R9-12) and (R13-16) are omitted because they closely resemble (R1-4) and (R5-8), respectively. The only difference lies in the specific products they target—products A and B—outlined at level 4. At level 4, requirements are split between the transportation of product A (R1-8) and product B (R9-16). At level 3, R1-8 are broken down into safety constraints (R1-4) and efficiency constraints (R5-8), with R9-16 following a similar division. Level 2 further categorizes R1-4 into environment safety (e.g., R1: avoid placing product A in passageway area 3) and product safety (e.g., R3: prevent taking product A to shipment area 2). R5-8 are divided into time constraints, such as R5: each delivery must be completed within seven timesteps, and redundant action constraints, for instance, R7: avoid exiting shipment area 1 while carrying product A.

In this scenario, we introduce an environmental change to trigger the adaptation: "the path between areas 3 and 5 is blocked by an obstacle." Consequently, even if the robot in area 3 executes `move_down`, it will only result in `arrived[1]` as its observation, indicating no change in robot position due to the obstacle.

5.1.2 Production Cell System

The second application is a production cell system for processing materials in a factory. Illustrated in Fig. 6-(b), this system includes a robot arm, a material tray, and three processing machines for baking, pressing, and drilling. The environment model captures the potential interactions among the robot arm, materials, and processing machines. For example, the robot arm's controllable actions include lifting materials and moving to specific locations, while the processing machines' controllable actions relate to their specific processing tasks. The monitorable outcomes include the success or failure of these processes and movement, with potential failures exemplified by scenarios such as material damage from repeated baking.

Figure 8 outlines the multi-grained requirement design for the production cell across four levels. At the top level, requirements are grouped into those associated with the robot arm and those for the processing machines. Level 3 further categorizes requirements into: (R1–2) handling rules, such as R1: the arm should not pick up materials if already holding something; (R3–10) movement rules for the robot arm; (R11–16) operational rules for each processing machine, like R11: bake each material only once; and (R17–18) processing orders, e.g., R18: pressing should follow baking and drilling. Level 2 divides R3-10 into location constraints (e.g., R3: the arm must perform a pick-up or put-down action when reaching the oven to avoid unnecessary movement) and action constraints (e.g., the arm should not pick up at the 'drill' location until the 'drill' operation has occurred). R11-16 are detailed into specific rules for baking, drilling, and pressing, such as R11: a material can only be baked once.

In this application, the environmental change that triggers adaptation is assumed to be due to a malfunction, where the robot arm may incorrectly move to the wrong location. This malfunction could result in unnecessary robot arm movements that were designed to be avoided by requirements.

Table 2 Experiment results of warehouse robot system. The reduction is compared with level 1.

Granularity Level	Number of Requirement	Analysis Time (ms)	Time Reduction	Game State	State Reduction	Adaptation Quality (max=16)
4	2	274	97.25%	4,745	99.96%	8
3	4	537	94.62%	14,985	99.55%	12
2	8	1,300	86.97%	86,408	97.40%	14
1	16	9,974	-	3,329,701	-	15

Table 3 Experiment results of production cell system.The reduction is compared with level 2.

Granularity Level	Number of Requirement	Analysis Time (ms)	Time Reduction	Game State	State Reduction	Adaptation Quality (max=16)
4	2	168	94.66%	4,071	98.78%	8
3	4	642	79.59%	43,503	86.91%	12
2	7	3.146	-	332,341	-	14
1	18	OutOfMemory Error				

5.2 Effectiveness – RQ2

5.2.1 Experiment Setting

To address RQ2, we measure the computation time of GRA and the adaptation quality at each granularity level. As a baseline for comparison, we assume the approach of one-time adaptation with a static granularity, as described in prior studies [5], [8], [16]. More specifically, we consider the requirements equivalent to the ones at the lowest level used in MGRA for consistency in the experiment.

Experiments were conducted on a desktop equipped with Intel W-2123 CPU and 32GB RAM. The program was developed in Java, with the Java Virtual Machine (JVM) heap memory allocated to a maximum of 2GB (Eclipse's default settings) for this experiment.

5.2.2 Experiment Results

Tables 2 and 3 summarize the experiment results, showing a distinct trend in the relationship between computation time and adaptation quality. From these results, several points can be drawn:

Firstly, focusing specifically on level 4 analysis time, it is evident that the MGRA facilitates swift initial adaptation by employing GRA at the highest granularity level. In the warehouse application, MGRA at level 4 significantly reduces the number of states in the analysis space and the analysis time by 99.96% and 97.25%, respectively, compared to the baseline at level 1. Similarly, in the production cell application, the number of states and computation time at level 4 are only 1.22% and 5.31% of those at level 2, respectively. These findings underscore that enhancing the granularity of analysis can substantially lower the time and space costs of computation, thereby mitigating the scalability challenges associated with GRA to a notable extent.

Secondly, another notable observation is the result at level 1 in Table 3, where the baseline approach fails to execute GRA due to an OutOfMemory Error. This outcome highlights an additional potential advantage of our proposal, that is, the increase of analysis granularity in MGRA allows for the application of GRA in memory-constrained environments, such as embedded systems.

Thirdly, since Tables 2 and 3 only give the computation time for each single adaptation, we further discuss the overall computation time of MGRA. For the overall computation time, its comparison varies based on the computation scenario: (i) If GRA at varying granularity levels is executed in parallel with the same computing resources as the baseline, then the computation time remains the same for both MGRA and the baseline; (ii) If GRA is executed sequentially from the highest to the lowest granularity level, MGRA demands more time than the baseline, with the warehouse and production cell needing 1.18 (calculated by $\frac{227+537+1300+9974}{9974}$) and 1.26 (calculated by $\frac{168+642+3146}{3146}$) times the baseline computation time, respectively.

5.3 Discussion

For RQ1, our application of MGRA in two extensively discussed scenarios in DCS studies has demonstrated its applicability. Throughout these scenarios, we experienced no issues in either design-time requirement modeling or runtime analysis execution.

As for RQ2, the experimental results indicate that GRA conducted at high granularity levels necessitates extremely little computation time (compared with baseline). This efficiency not only enhances the responsiveness of adaptation but also avoids the infeasibility of low-granularity analysis due to memory constraints. However, due to the nature of GRA, as granularity decreases, the time needed to enhance adaptation quality increases exponentially.

Additionally, a topic pertinent to both RQs 1 and 2 concerns the conditions where increasing the analysis granularity does not reduce computation costs, which is crucial to determine how much adaptation timeliness the highest granularity-level analysis can offer. Theoretically, as discussed in Sect. 3, increasing analysis granularity fundamentally abstracts the state combination explosion, which aims to analyze the satisfiability of multiple requirements, to only analyze the satisfiability of a single AND-composited require-

ment. This essence indicates that increasing the granularity of analysis invariably mitigates state explosion. An exception occurs in the rare case where the state combination explosion does not happen, such as when two requirements are exactly or almost identical. In such cases, AND-compositing the requirements may not reduce the computation costs.

From a practical perspective, another critical consideration for designing MGRA is how to balance the trade-off between computation time and adaptation quality at different levels throughout the entire MGRA process. A relevant metric for evaluating such a trade-off could be the integral of adaptation quality over time, calculated from the start of MGRA to its completion (the bigger the better). Using sequential computation in the warehouse application as an example, the integral result for MGRA is 159,532, calculated by summing the products of computation times and corresponding adaptation quality at each level: $274 \times 0 + 537 \times 8 + 1,300 \times 12 + 9,974 \times 14$. Here, 274 represents the computation time at level 1, and 0 indicates that the system may suspend until the adaptation solution is generated, so the adaptation quality during such a period is 0. In contrast, the integral result for the baseline is 31,665, calculated by $9,974 \times 0 + (274 + 537 + 1,300) \times 15$, where $274+537+1,300$ refers to the difference in total computation time between MGRA and the baseline.

5.4 Limitations

The main limitations are as follows: Firstly, for simplicity, we treat all requirements at the lowest level as having equal weight, although actual weights could be more nuanced, such as composite or dynamic weights. This simplification is made because our paper primarily addresses the computation of overhead GRA, which does not depend on the configuration of weights. For more complicated weight configurations, we can refer to QoS (Quality of Service) design [22], or dynamic weight adjustment methods [18]. Secondly, in line with existing GRA studies, we simplify requirement modeling by considering all requirements as mutually independent. However, interdependencies between requirements—where the satisfaction of one may hinge on another—exist. Future research will explore methodologies for integrating such dependencies into multi-grained modeling at the design-time phase. Incorporating this type of dependency can potentially reduce the computational cost of GRA; for example, if the prerequisite requirements are not met, there is no need to analyze the subsequent requirements. Thirdly, for simplicity in our experiment, we assumed that all requirements could be relaxed. However, in real scenarios, certain requirements might be mandatory. For instance, in a production cell scenario, repeated baking could result in accidents, necessitating some requirements to be strictly satisfied. In such cases, it's crucial for designers to devise additional logic to determine whether an urgent system halt is needed based on GRA outcomes. It's worth mentioning that MGRA remains beneficial here by amalgamating all mandatory requirements into a single unit to expedite the analysis process. Fourthly, our approach currently presupposes that specification swaps occur only when the system reverts to its initial state (e.g., the robot moves back to the initial location after finishing one delivery), potentially introducing extra swapping overhead. This assumption is due to current constraints in runtime dynamic update techniques [19], which require manual efforts involved in remapping pre- and post-update specifications. Future efforts will aim at automating this remapping process to facilitate fully automatic dynamic updates, thus mitigating the swapping overhead.

5.5 Threats to Validity

The primary validity threat to MGRA pertains to external validity. Firstly, the observed reduction in computation time is directly linked to the requirement design; that is, the computation time at each granularity level depends on the requirements designed for that particular level. Secondly, while the focus of this paper is not on the environment model, the modeling of the environment–especially the abstraction level of its labels–will also significantly affect the computation time required for GRA.

6. Related Work

6.1 Runtime Requirement Adaptation

Adapting requirements in response to environmental uncertainties has been a focal point of research, with various methods developed to address different facets of requirements management. Literature [23], centers on refining relationships between requirements, and presents a technique for dynamically adjusting the goal model according to environmental changes. Literature [24], targets requirement priorities, introduces a method to modify priorities based on historical execution data. Similarly, literature [18] explores the adjustment of requirement weights through a simulation method that incorporates user feedback and complaints. Diverging from these approaches, our study places emphasis on the satisfiability of requirements. We approach this by relaxing unsatisfiable or conflicting requirements after checking their satisfiability.

6.2 Design-Time Approach for Requirement Relaxation

Development-time approaches necessitate that domain experts and system designers specify the logic of requirement relaxation (e.g., which requirement to relax under certain situations) during the development phase [25]. Among the most notable methods in this context are fault-tolerant design [26] and graceful degradation [27], which have seen extensive application in various real-world scenarios.

Additionally, the requirement specification languages like RELAX [28] and FLAGS [29] have been developed. These languages enable the definition of relaxation operators and fuzzy goals to aid in designing requirement relaxation strategies. However, due to the associated costs and

the unpredictability of changes that can occur at runtime, it is often impractical for designers to preemptively enumerate relaxation logic for all conceivable changes.

6.3 Runtime Approach for Requirement Relaxation

To address the limitations of the development-time approach, which only functions in pre-defined situations, runtime approaches have been proposed to enable systems to autonomously decide how to relax requirements. Early studies did not consider relaxation, focusing instead on using verification techniques to identify states that violated requirements. These states were then excluded to generate a controller that could guarantee requirement satisfaction [30]. Subsequent studies, such as [4], treated losing regions in a binary manner. This meant that each analysis could only check the feasibility of a single requirement combination, necessitating multiple analyses for different combinations. Aizawa et al. introduced the concept of PLR, enabling the feasibility of all requirement combinations to be checked in a single analysis (i.e., GRA) [16]. Building on this, Li et al. proposed a difference analysis algorithm that focuses on the changed parts of the game to enhance the efficiency of GRA [5], [31]. Despite attempts by literature [5], [31] to reduce computation time for runtime applicability by focusing only on the changes in PLR, they do not fundamentally address the core issues of computation time, namely state explosion and combination explosion. This paper addresses these challenges by increasing the granularity of the analysis, making it feasible to apply GRA to larger-scale systems, albeit with a trade-off in adaptation quality.

7. Conclusion and Future Work

This paper introduces the Multi-grained Guaranteeable Requirement Analysis (MGRA) to address the scalability challenges inherent in GRA. MGRA leverages the trade-off between computation time and adaptation quality, enabling an iterative adaptation process. Initially, MGRA begins with a coarse-grained GRA, which offers quicker but lower-quality adaptation solutions to mitigate the scalability issue and meet the demands for timely adaptation. Subsequently, MGRA incrementally improves the adaptation quality by lowering the analysis granularity of GRA. Experimental results from two case studies illustrate that MGRA is capable of providing an initial adaptation solution within a significantly shorter time window (i.e., reductions of 97.25% and 94.66%), with subsequent gradual improvements in adaptation quality.

Looking ahead, our future work will focus on enhancing MGRA in two critical aspects: First, we aim to expand MGRA to include support for requirement dependencies, such as cases where the fulfillment of one requirement is a precondition for another. This expansion aims to extend the applicability of MGRA to more complex scenarios [32], [33], as well as potentially further reducing computational costs. Second, we intend to investigate ways to automate the dynamic update during operation, enabling more efficient specification swapping when applying MGRA.

Acknowledgments

The research was partially supported by JSPS KAKENHI (grant number 23H03374, and 22KJ2935).

References

[1] J. Kramer, "Re @ runtime : the challenge of change re'20 conference keynote," 2020 IEEE 28th International Requirements Engineering Conference (RE), pp.4–6, 2020.
[2] D. Sykes, W. Heaven, J. Magee, and J. Kramer, "From goals to components: A combined approach to self-management," Proc. 2008 International Workshop on Software Engineering for Adaptive and Self-Managing Systems, SEAMS '08, pp.1–8, 2008.
[3] Lori MacVittie, "How to gracefully degrade web 2.0 applications to maintain availability." https://web.archive.org/web/20100428043002/http://devcentral.f5.com/weblogs/macvittie/archive/2010/01/27/how-to-gracefully-degrade-web-2.0-applications-to-maintain-availability.aspx. Accessed: 2024-01-30.
[4] N. D'Ippolito, V. Braberman, J. Kramer, J. Magee, D. Sykes, and S. Uchitel, "Hope for the best, prepare for the worst: Multi-tier control for adaptive systems," Proc. 36th International Conference on Software Engineering, ICSE 2014, p.688–699, 2014.
[5] J. Li, K. Aizawa, K. Tei, and S. Honiden, "Efficient difference analysis algorithm for runtime requirement degradation under system functional fault," 2020 IEEE 18th International Conference on Embedded and Ubiquitous Computing (EUC), pp.33–40, 2020.
[6] J. Li and K. Tei, "Done is better than perfect: Iterative adaptation via multi-grained requirement relaxation," 2022 IEEE 30th International Requirements Engineering Conference (RE), pp.288–294, 2022.
[7] H. Shi, W. Dong, R. Li, and W. Liu, "Controller resynthesis for multirobot system when changes happen," Computer, vol.53, no.12, pp.69–79, 2020.
[8] T. Hirano, K. Tei, K. Aizawa, and S. Honiden, "Differential controller synthesis at runtime using changed parts of environment model," 2021 IEEE 8th International Conference on Industrial Engineering and Applications (ICIEA), pp.91–100, 2021.
[9] P. Zave and M. Jackson, "Four dark corners of requirements engineering," ACM Trans. Softw. Eng. Methodol., vol.6, no.1, pp.1–30, Jan. 1997.
[10] J. Li, T. Yamauchi, N. Li, Z. Chen, M. Zhang, T. Hirano, and K. Tei, "Demonstration of a real-world self-adaptive robot path-finding using discrete controller synthesis," 4th IEEE International Conference on Autonomic Computing and Self-Organizing Systems, 2023.
[11] D. Giannakopoulou and J. Magee, "Fluent model checking for event-based systems," SIGSOFT Softw. Eng. Notes, vol.28, no.5, pp.257–266, Sept. 2003.
[12] B. Alpern and F.B. Schneider, "Recognizing safety and liveness," Distributed Computing, vol.2, no.3, pp.117–126, 1987.
[13] V. Braberman, N. D'Ippolito, N. Piterman, D. Sykes, and S. Uchitel, "Controller synthesis: From modelling to enactment," Proc. 2013 International Conference on Software Engineering, ICSE '13, pp.1347–1380, IEEE Press, 2013.
[14] J. Magee and J. Kramer, Concurrency: State Models and Java Programs, 2nd ed., Wiley Publishing, 2006.
[15] S.J. Russell and P. Norvig, Artificial Intelligence: a modern approach, 3 ed., Pearson, 2009.
[16] K. Aizawa, K. Tei, and S. Honiden, "Identifying safety properties guaranteed in changed environment at runtime," 2018 IEEE International Conference on Agents (ICA), pp.75–80, 2018.
[17] K. Aizawa, K. Tei, and S. Honiden, "Analysis space reduction with state merging for ensuring safety properties of self-adaptive systems," 2019 IEEE SmartWorld, Ubiquitous Intelligence Computing,

Advanced Trusted Computing, Scalable Computing Communications, Cloud Big Data Computing, Internet of People and Smart City Innovation, pp.1363–1370, Aug. 2019.

[18] N. Li, M. Zhang, J. Li, E. Kang, and K. Tei, "Preference adaptation: user satisfaction is all you need!," 2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp.133–144, 2023.

[19] L. Nahabedian, V. Braberman, N. D'Ippolito, S. Honiden, J. Kramer, K. Tei, and S. Uchitel, "Dynamic update of discrete event controllers," IEEE Trans. Softw. Eng., vol.46, no.11, pp.1220–1240, Nov. 2020.

[20] J.O. Kephart and D.M. Chess, "The vision of autonomic computing," Computer, vol.36, no.1, pp.41–50, Jan. 2003.

[21] D. Sykes, D. Corapi, J. Magee, J. Kramer, A. Russo, and K. Inoue, "Learning revised models for planning in adaptive systems," 2013 35th International Conference on Software Engineering (ICSE), pp.63–71, May 2013.

[22] W. Qian, X. Peng, B. Chen, J. Mylopoulos, H. Wang, and W. Zhao, "Rationalism with a dose of empiricism: Case-based reasoning for requirements-driven self-adaptation," 2014 IEEE 22nd International Requirements Engineering Conference (RE), pp.113–122, 2014.

[23] D. Alrajeh, A. Cailliau, and A. van Lamsweerde, "Adapting requirements models to varying environments," the ACM/IEEE 42nd International Conference on Software Engineering (ICSE), pp.50–61, 2020.

[24] H. Samin, L.H.G. Paucar, N. Bencomo, and P. Sawyer, "Towards priority-awareness in autonomous intelligent systems," Proc. 36th Annual ACM Symposium on Applied Computing, pp.1328–1337, 2021.

[25] A. van Lamsweerde, Requirements Engineering: From System Goals to UML Models to Software Specifications, 1st ed., Wiley Publishing, 2009.

[26] L.L. Pullum, Software Fault Tolerance Techniques and Implementation, Artech House, Inc., USA, 2001.

[27] T. Saridakis, Design Patterns for Graceful Degradation, pp.67–93, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.

[28] J. Whittle, P. Sawyer, N. Bencomo, B.H. Cheng, and J.M. Bruel, "Relax: Incorporating uncertainty into the specification of self-adaptive systems," 2009 17th IEEE International Requirements Engineering Conference, pp.79–88, 2009.

[29] L. Baresi, L. Pasquale, and P. Spoletini, "Fuzzy goals for requirements-driven adaptation," 2010 18th IEEE International Requirements Engineering Conference, pp.125–134, 2010.

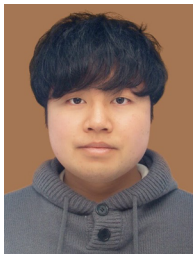
[30] A. Borda, L. Pasquale, V. Koutavas, and B. Nuseibeh, "Compositional verification of self-adaptive cyber-physical systems," 2018 IEEE/ACM 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp.1–11, May 2018.

[31] J. Li, K. Tei, and S. Honiden, "Identifying achievable goals for adaptive replanning against runtime environment change," Intelligent Systems Design and Applications, Cham, pp.945–955, Springer International Publishing, 2021.

[32] J. Li, W. Manzano, T. Yamauchi, N. Matsuyama, E.Y. Nakagawa, and K. Tei, "Employing discrete controller synthesis for developing systems-of-systems controllers," 2024 12th ACM/IEEE International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems (SESoS@ICSE), pp.1–8, April 2024.

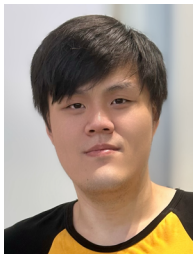
[33] T. Yamauchi, T. Hirano, J. Li, T. Kawasaki, Y. Chen, A. Tsuge, T. Okoshi, J. Nakazawa, N. Yoshioka, G. Palaiokrassas, A. Litke, and K. Tei, "A development method for safe node-red systems using discrete controller synthesis," 2021 IEEE International Conferences on Internet of Things (iThings), pp.130–137, 2021.

Jialong Li is now a Ph.D. student at Waseda University in Tokyo, Japan. He received B.E and M.E degrees in Computer Science from Waseda University in 2019 and 2021. His research interests are in self-adaptive systems, and requirement engineering. He is a member of the IEEE Computer Society (IEEE CS), and Association for Computing Machinery (ACM). He has served Program Committee of International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS).



Takuto Yamauchi works as an assistant professor in the Department of Computer Science and Communications Engineering at Waseda University. He received the degree of Doctor of Engineering from Waseda University in 2024. His research focuses on self-adaptive systems and the controller synthesis for discrete event systems.

Takanori Hirano is now a doctoral student at Waseda University in Tokyo, Japan. He received B.E and M.E degrees in Computer Science from Waseda University in 2019 and 2021. His research interests are in discrete controller synthesis.



Jinyu Cai is now a Ph.D. student at Waseda University in Tokyo, Japan. He received M.E degrees in Computer Science from Waseda University in 2023. He is now funded by JST SPRING (Japan Science and Technology Agency, Support for Pioneering Research Initiated by the Next Generation). His research interests lie in deep reinforcement learning and adaptive systems.

Kenji Tei is an associate professor at Tokyo Institute of Technology. He is also an associate professor by special appointment of National Institute of Informatics (NII). He is interested in self-adaptive software, software architecture, model-driven engineering, and software verification. He is a member of the IEEE Computer Society (IEEE CS), Association for Computing Machinery (ACM), Information Processing Society of Japan (IPSJ), and Institute of Electronics, Information and Communication Engineers (IEICE). He currently serves Associate Editor of ACM Transactions on Autonomous and Adaptive Systems (TAAS). He has served Steering Committee Member of International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS) and Program co-Chair of SEAMS '19.